



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Laboratorio di
Sicurezza Informatica

Attacchi al processo di autenticazione e autorizzazione

Marco Prandini

Dipartimento di Informatica – Scienza e Ingegneria

Una volta entrati

L'accesso come utente legittimo dà solitamente privilegi molto limitati

Ci sono diverse tecniche per elevare questi privilegi sfruttando vulnerabilità locali

Richiamiamo rapidamente

- i meccanismi di autorizzazione di Linux
- i processi di sistema

le cui vulnerabilità possono portare alla **privilege escalation**

Bombe logiche

Ci sono diversi processi eseguiti con privilegi di amministrazione

Se i file usati da tali processi hanno permessi errati, un utente non privilegiato potrebbe *iniettare* il proprio codice e attendere che il sistema lo esegua

- in un momento predeterminato
- all'avvio o arresto di un sottosistema
- al verificarsi di un evento

Vediamo quali sono questi sistemi e dove sono i corrispondenti file

Esecuzione periodica - cron

crond è un demone che esamina una serie di file di configurazione ogni minuto, e determina quali compiti specificati nei file debbano essere eseguiti.

I file di configurazione (**crontab**) sono distinti in due insiemi:

- Uno per utente (**/var/spool/cron/crontabs/<utente>**)

- si visualizza / edita / sostituisce con

crontab -l / crontab -e / crontab <nuova_tab>

- System-wide (**/etc/crontab**)

- Solitamente quest'ultimo non fa altro che richiamare l'esecuzione di tutto ciò che trova in alcune directory:

**/etc/cron.hourly/
/etc/cron.daily/
/etc/cron.weekly/
/etc/cron.monthly/**

ha un campo in più rispetto ai file personali per indicare a nome di che utente eseguire ogni task configurato

Esecuzione periodica - cron

Ogni crontab contiene un elenco di direttive nella forma

MINUTO ORA G.MESE MESE G.SETTIMANA <comando>

Es.

*	*	27	*	*	\$HOME/bin/paga
30	8-18/2	*	*	1-5	\$HOME/bin/lavora
00	00	1	1	*	/usr/sbin/auguri
30	4	1,15	*	6	/bin/backup

L'azione è eseguita quando l'ora corrente corrisponde a tutti i selettori di una riga (campi in AND logico)

ECCEZIONE: se sono specificati (diversi da *) entrambi i giorni (settimana e mese), i due campi sono considerati in OR logico

- nell'esempio, il backup viene eseguito ogni mese il giorno **1** + il giorno **15** + ogni **sabato**, alle 4:30

Esecuzione posticipata - at

atd è un demone che gestisce code di compiti da svolgere in momenti prefissati. L'interfaccia ad *atd* consiste di 4 comandi:

at [-V] [-q queue] [-f file] [-mldbv] TIME
pianifica un comando al tempo TIME

atq [-V] [-q queue] [-v]
elenca i comandi in coda

atrm [-V] job [job...]
rimuove comandi dalla coda

batch [-V] [-q queue] [-f file] [-mv] [TIME]
esecuzione condizionata al carico

Esecuzione posticipata - at

Se non viene specificato un file comandi per at o batch, verrà usato lo standard input.

La specifica dell'ora è flessibile e complessa. Per una definizione completa si veda la documentazione in

[`/usr/share/doc/at/timespec`](#)

Alcuni esempi:

```
echo 'wall "sveglia"' | at 08:00
```

```
echo "$HOME/bin/pulisci" | at now + 2 weeks
```

```
echo "$HOME/bin/auguri" | at midnight 31.12.2021
```

Event managers / IPC systems

Dbus è un'architettura di Inter-Process Communication

- Nata per uniformare la comunicazione tra elementi delle interfacce desktop
- Curiosate in `/etc/dbus-1/` per vedere i file di configurazione
- In `/etc/dbus-1/event.d` sono collocati gli script di avvio dei sottosistemi gestiti

Udev ha rimpiazzato devfs come **event manager** per la creazione istantanea dei device special file quando un nuovo dispositivo viene connesso; ora è parte di systemd

- In `/etc/udev/rules.d` sono configurate le regole evento → azione
- Es. `70-persistent-net.rules`
PCI device 0x10ec:0x8168 (r8169)
`SUBSYSTEM=="net", ACTION=="add", DRIVERS=="?*", ATTR{address}`
`=="d0:67:e5:18:d9:e4", ATTR{dev_id}=="0x0", ATTR{type}=="1", KERNEL=="eth*",`
`NAME="eth0"`

alla comparsa nel subsystem `net` di una scheda con `MAC=d0:67:e5:18:d9:e4` le assegna nome `eth0`

Inizializzazione e attività in background (demoni)

init è il primo processo avviato dal kernel

- Gestisce i *runlevel*
 - stati di funzionamento del sistema definiti dal sottoinsieme di servizi attivi
- Orchestra la sequenza corretta di eventi per raggiungere un runlevel
- Intercetta e gestisce alcuni eventi
 - es. ctrl-alt-canc, terminazione anomala di processi,
- Spegne il sistema in modo ordinato

Tre varianti principali

- (storico) SystemV-style initialization
- Upstart (Canonical, 2006-2014)
- Systemd (ispirazione RedHat, 2010-oggi)

utile da conoscere
per l'attuale mix
imprevedibile di
distribuzioni moderne
e tradizionali

sysvinit

`/sbin/init` dell'originale SystemV Unix

- configurato dal file `/etc/inittab`
- *inittab* specifica il default runlevel
 - `id:2:initdefault:`
- ma se la keyword `single` viene passata come parametro al kernel dal boot loader, questo settaggio è scavalcato e il sistema parte in *single user mode* (runlevel 1)
 - `~~:S:wait:/sbin/sulogin`
- *init* avvia i virtual terminal e i gestori delle console su linea seriale (può sembrare un arcaismo, ma nel mondo IoT è tornato alla ribalta)
 - `1:2345:respawn:/sbin/getty 38400 tty1`
 - `2:23:respawn:/sbin/getty 38400 tty2`
 - ...
 - `T0:23:respawn:/sbin/getty -L ttyS0 9600 vt100`
 - `T3:23:respawn:/sbin/mgetty -x0 -s 57600 ttyS3`

processi avviati da sysvinit

init è in senso astratto responsabile per tutti i processi che girano sul sistema, ma in particolare due attività possono essere direttamente ricondotte ad esso

– linee del tipo

```
10:0:wait:/etc/init.d/rc 0
```

pilotano il processo di avvio *System-V-style*

- wait = esecuzione sequenziale
- quando il runlevel obiettivo è 'N', **rc** esegue
 - ogni programma con nome che inizia per 'S' in **/etc/rcN.d/** col parametro **start**
 - ogni programma con nome che inizia per 'K' in **/etc/rcN.d/** col parametro **stop**
- per evitare l'inutile duplicazione degli script di avvio e arresto dei demoni, questi sono tutti raccolti in **/etc/init.d/**, e symlinked dalle 7 directory **/etc/rcN.d/**

– linee del tipo

```
x:5:respawn:/usr/X11/bin/gdm
```

avviano il programma specificato come 4° campo, e *init* monitora il processo per riavviarlo (respawn) se termina

Systemd – termini

Diversi tipi di **[control] unit** i cui nomi seguono la convenzione **name.type**

type può essere:

- **Service**: controllo e monitoraggio dei demoni
- **Socket**: attivazione di canali IPC di ogni tipo (file, net socket, Unix socket)
- **Target**: gruppo di unit che **rimpiazza il concetto di runlevel**
- **Device**: punti di accesso ai dispositivi, creati dal kernel in seguito a interazioni con l'hardware
 - filesystem-related: **Mounts**, **Automounts**, **Swap**
- **Snapshots**: stato salvato del sistema
- **Timers**: attività legate al tempo (→ cron, at)
- **Paths**: monitoraggio del contenuto di una directory via inotify
- **Slices**: gestione delle risorse via cgroup
- **Scopes**: raggruppamento di processi per miglior organizzazione

Systemd – dove trovare le definizioni delle unit

“libreria” di definizioni di riferimento

- `/lib/systemd/system/`

File forniti dai mantainer dei diversi pacchetti software

- Quasi sempre link alle definizioni di riferimento
- `/usr/lib/systemd/system/`

File con le personalizzazioni

- prioritari rispetto alle definizioni di sistema sopra elencate
- `/etc/systemd/system`

Systemd – avvio e arresto dei servizi

Controllo a run time dei servizi

- `systemctl {start|stop|status|restart|reload} servicename`
 - ...intuitivo
 - output molto descrittivo dello stato
 - stato corrente ed elenco dei passi fatti per raggiungerlo
 - process tree
 - righe di log rilevanti
 - con `-H [hostname]` si connette a un host remoto via ssh

Cosa fanno in realtà? Vedremo in pratica i dettagli, ma in sintesi

- Nelle unit sono definiti i comandi da eseguire attraverso parametri di configurazione (`ExecStart`, `ExecReload`, `ExecStop`)
 - `restart` è semplicemente `stop` seguito da `start`
- systemd tiene traccia dei processi avviati con `start`, in modo che i loro PID possano essere usati come parametri nei comandi `reload/stop`
- `stop`, oltre a eseguire (l'eventuale) comando specificato, di default manda `SIGTERM` al processo, seguito da `SIGKILL` dopo un timeout. Moltissime varianti configurabili: `man 5 systemd.kill`

Systemd – boot e shutdown

Le operazioni descritte alla slide precedente sono **volatili**

- si impartisce il comando
- si ottiene l'effetto
- nulla cambia nella **configurazione** del sistema

Per automatizzare avvio al boot e arresto allo shutdown si utilizzano invece

- `systemctl {enable|disable|mask|unmask} servicename`
 - **disable** lascia disponibile la possibilità di usare manualmente **start**
 - **mask** "neutralizza" l'intera definizione della unit, impedendo anche il controllo manuale
- questi comandi non hanno alcun effetto immediato
- **l'effetto sulla configurazione del sistema è persistente**